

Seção 19.15 Construtores explicit

- C++ oferece a palavra-chave `explicit` para suprimir conversões implícitas por meio de construtores de conversão quando essas conversões não são permitidas. Um construtor que é declarado como `explicit` não pode ser usado em uma conversão implícita.

Seção 19.16 Classes proxy

- Fornecer aos clientes de sua classe uma classe proxy que conheça apenas a interface `public` de sua classe permite que os clientes usem os serviços de sua classe sem dar aos clientes acesso aos detalhes de implementação de sua classe, como seus dados `private`.
- Quando uma definição de classe usa apenas um ponteiro ou uma referência a um objeto de outra classe, o arquivo de

cabeçalho da classe para essa outra classe (que normalmente revelaria os dados `private` dessa classe) não precisa ser incluído com `#include`. Você pode simplesmente declarar essa outra classe como um tipo de dado com uma declaração de classe direta antes que o tipo seja usado no arquivo.

- O arquivo de implementação que contém as funções-membro de uma classe proxy é o único arquivo que inclui o arquivo de cabeçalho para a classe cujos dados `private` queremos ocultar.
- O arquivo de implementação que contém as funções-membro da classe proxy é fornecido ao cliente como um arquivo de código objeto pré-compilado, com o arquivo de cabeçalho que inclui protótipos de função dos serviços fornecidos pela classe proxy.

Terminologia

alocar 576

armazenamento livre 576

autoatribuição 586

classe proxy 600

construtor de cópia 583

construtores de argumento único 587

construtores de conversão 587

declaração de classe direta 601

`delete`, operador 576

`delete[]`, operador 576

desalocar 576

`empty`, função-membro de `string` 597

`explicit`, construtor 598

função-membro `at` da classe `string` 597

função operador de cast 588

gerenciamento dinâmico de memória 576

heap 576

inicializador 576

`new`, operador 576

`new[]`, operador 576

operador de chamada de função `()` 589

operador de conversão 588

ponteiro suspenso 585

sobrecarga de operador 568

`substr`, função-membro da classe `string` 597

vazamento de memória 576

Exercícios de autorrevisão

19.1 Preencha os espaços em cada uma das sentenças:

- Suponha que `a` e `b` sejam variáveis inteiras e formemos a soma `a + b`. Agora, suponha que `c` e `d` sejam variáveis de ponto flutuante e formemos a soma `c + d`. Os dois operadores `+` estão claramente sendo usados para propósitos diferentes. Este é um exemplo de _____.
- A palavra-chave _____ apresenta uma definição de função operador sobrecarregada.
- Para usar operadores em objetos de classe, eles precisam ser sobrecarregados, com a exceção dos operadores de _____, _____ e _____.
- _____, _____ e _____ de um operador não podem ser mudadas pela sobrecarga do operador.

e) Os operadores que não podem ser sobrecarregados são _____, _____, _____ e _____.

f) O operador _____ reivindica a memória alocada anteriormente por `new`.

g) O operador _____ aloca memória dinamicamente para um objeto de um tipo especificado e retorna um(a) _____ para esse tipo.

19.2 Explique os vários significados dos operadores `<<` e `>>`.

19.3 Em que contexto o nome `operator/` poderia ser usado?

19.4 (Verdadeiro/Falso) Somente operadores existentes podem ser sobrecarregados.

19.5 Qual é a diferença entre a precedência de um operador sobrecarregado e a precedência do operador original?

■ Respostas dos exercícios de autorrevisão

- 19.1** a) sobrecarga de operador. b) operator. c) atribuição (=), endereço (&), vírgula (,). d) precedência, associatividade, 'aridade'. e) ., ?:, .* e ::. f) delete. g) new, ponteiro.
- 19.2** O operador >> é tanto o operador de deslocamento à direita quanto o operador de extração de stream, a depender de seu contexto. O operador << é tanto o operador de deslocamento à esquerda quanto o operador de inserção de stream, a depender de seu contexto.
- 19.3** Para a sobrecarga de operador: ele seria o nome de uma função que ofereceria uma versão sobrecarregada do operador / para uma classe específica.
- 19.4** Verdadeiro.
- 19.5** A precedência é idêntica.

■ Exercícios

- 19.6** Compare os operadores de alocação e desalocação de memória dinâmica new, new [], delete e delete [].
- 19.7** **Sobrecarga do operador de parênteses.** Um bom exemplo da sobrecarga do operador de chamada de função () é permitir outra forma de subscrito de array duplo, popular em algumas linguagens de programação. Em vez de dizer

```
tabuleiroXadrez[ linha ][ coluna ]
```

para um array de objetos, sobrecarregue o operador de chamada de função para permitir a forma alternativa

```
tabuleiroXadrez( linha, coluna )
```

Crie uma classe DoubleSubscriptedArray que tenha recursos semelhantes à classe Array nas figuras 19.6 e 19.7. No momento da construção, a classe deverá ser capaz de criar um array de qualquer número de linhas e qualquer número de colunas. A classe deverá fornecer operator() para realizar operações de duplo subscrito. Por exemplo, em um DoubleSubscriptedArray de 3 por 5 chamado a, o usuário poderia escrever a(1, 3) para acessar o elemento na linha 1 e coluna 3. Lembre-se de que operator() pode receber qualquer número de argumentos. A representação básica do array de duplo subscrito deve ser um array de único subscrito de inteiros números de *linhas* * *colunas* de elementos. A fun-

ção operator() deverá realizar a aritmética de ponteiro apropriada para acessar cada elemento do array. Deverá haver duas versões de operator() — uma que retorne int & (de modo que um elemento de um DoubleSubscriptedArray possa ser usado como um lvalue) e uma que retorne const int &. A classe também deve oferecer os seguintes operadores: ==, !=, =, << (para enviar o array no formato de linha e coluna) e >> (para receber o conteúdo inteiro do array).

- 19.8** **Classe Complex.** Considere a classe Complex mostrada nas figuras 19.20 a 19.22. A classe permite operações nos chamados *números complexos*. Estes são números no formato $\text{realPart} + \text{imaginaryPart} * i$, onde i tem o valor

$$\sqrt{-1}$$

- a)** Modifique a classe para permitir a entrada e a saída de números complexos por meio de operadores >> e << sobrecarregados, respectivamente (você deverá remover a função print da classe).
- b)** Sobrecarregue o operador de multiplicação para permitir a multiplicação de dois números complexos, como na álgebra.
- c)** Sobrecarregue os operadores == e != para permitir comparações de números complexos.

```
1 // Figura 19.20: Complex.h
2 // Definição da classe Complex.
3 #ifndef COMPLEX_H
4 #define COMPLEX_H
5
6 class Complex
7 {
8 public:
9     Complex( double = 0.0, double = 0.0 ); // construtor
10     Complex operator+( const Complex & ) const; // adição
11     Complex operator-( const Complex & ) const; // subtração
12     void print() const; // saída
```

Figura 19.20 ■ Definição da classe Complex. (Parte I de 2.)

```

13 private:
14     double real; // parte real
15     double imaginary; // parte imaginária
16 }; // fim da classe Complex
17
18 #endif

```

Figura 19.20 ■ Definição da classe Complex. (Parte 2 de 2.)

```

1 // Figura 19.21: Complex.cpp
2 // Definições de função-membro da classe Complex.
3 #include <iostream>
4 #include "Complex.h" // Definição da classe Complex
5 using namespace std;
6
7 // Construtor
8 Complex::Complex( double realPart, double imaginaryPart )
9     : real( realPart ),
10     imaginary( imaginaryPart )
11 {
12     // corpo vazio
13 } // fim do construtor de Complex
14
15 // operador de adição
16 Complex Complex::operator+( const Complex &operand2 ) const
17 {
18     return Complex( real + operand2.real,
19                     imaginary + operand2.imaginary );
20 } // fim da função operator+
21
22 // operador de subtração
23 Complex Complex::operator-( const Complex &operand2 ) const
24 {
25     return Complex( real - operand2.real,
26                     imaginary - operand2.imaginary );
27 } // fim da função operator-
28
29 // exibe um objeto Complex na forma: (a, b)
30 void Complex::print() const
31 {
32     cout << '(' << real << ", " << imaginary << ')';
33 } // fim da função print

```

Figura 19.21 ■ Definições de função-membro da classe Complex.

```

1 // Figura 19.22: fig19_22.cpp
2 // Programa de teste da classe Complex.
3 #include <iostream>
4 #include "Complex.h"
5 using namespace std;
6
7 int main()
8 {
9     Complex x;
10    Complex y( 4.3, 8.2 );
11    Complex z( 3.3, 1.1 );

```

Figura 19.22 ■ Números complexos. (Parte I de 2.)

```

12
13     cout << "x: ";
14     x.print();
15     cout << "\ny: ";
16     y.print();
17     cout << "\nz: ";
18     z.print();
19
20     x = y + z;
21     cout << "\n\nx = y + z:" << endl;
22     x.print();
23     cout << " = ";
24     y.print();
25     cout << " + ";
26     z.print();
27
28     x = y - z;
29     cout << "\n\nx = y - z:" << endl;
30     x.print();
31     cout << " = ";
32     y.print();
33     cout << " - ";
34     z.print();
35     cout << endl;
36 } // fim do main

```

```

x: (0, 0)
y: (4.3, 8.2)
z: (3.3, 1.1)
x = y + z:
(7.6, 9.3) = (4.3, 8.2) + (3.3, 1.1)
x = y - z:
(1, 7.1) = (4.3, 8.2) - (3.3, 1.1)

```

Figura 19.22 ■ Números complexos. (Parte 2 de 2.)

19.9 Classe HugeInt. Uma máquina com inteiros de 32 bits pode representar inteiros no intervalo de aproximadamente -2 bilhões a $+2$ bilhões. Essa restrição de tamanho fixo raramente é problemática, mas existem aplicações em que gostaríamos de poder usar um intervalo de inteiros muito maior. Foi para isso que C++ foi concebida, ou seja, para criar novos tipos de dados poderosos. Considere a classe HugeInt das figuras 19.23 a 19.25. Estude a classe cuidadosamente e depois resolva as seguintes questões:

a) Descreva exatamente como ela opera.

b) Que restrições a classe tem?

c) Sobrecarregue o operador de multiplicação $*$.

d) Sobrecarregue o operador de divisão $/$.

e) Sobrecarregue todos os operadores relacionais e de igualdade.

[Nota: não mostramos um operador de atribuição nem um construtor de cópia para a classe HugeInteger, pois o operador de atribuição e o construtor de cópia fornecidos pelo compilador são capazes de copiar o dado-membro de array inteiro corretamente.]

```

1 // Figura 19.23: Hugeint.h
2 // Definição da classe HugeInt.
3 #ifndef HUGEINT_H
4 #define HUGEINT_H
5
6 #include <iostream>

```

Figura 19.23 ■ Definição da classe HugeInt. (Parte 1 de 2.)

```

7  #include <string>
8  using namespace std;
9
10 class HugeInt
11 {
12     friend ostream &operator<<( ostream &, const HugeInt & );
13 public:
14     static const int digits = 30; // máximo de dígitos em um HugeInt
15
16     HugeInt( long = 0 ); // construtor de conversão/default
17     HugeInt( const string & ); // construtor de conversão
18
19     // operador de adição; HugeInt + HugeInt
20     HugeInt operator+( const HugeInt & ) const;
21
22     // operador de adição; HugeInt + int
23     HugeInt operator+( int ) const;
24
25     // operador de adição;
26     // HugeInt + string que representa valor inteiro grande
27     HugeInt operator+( const string & ) const;
28 private:
29     short integer[ digits ];
30 }; // fim da classe HugeInt
31
32 #endif

```

Figura 19.23 ■ Definição da classe HugeInt. (Parte 2 de 2.)

```

1  // Figura 19.24: Hugeint.cpp
2  // Definições de função-membro e função friend HugeInt.
3  #include <cctype> // protótipo de função isdigit
4  #include "Hugeint.h" // Definição de classe HugeInt
5  using namespace std;
6
7  // construtor default; construtor de conversão que converte um
8  // inteiro long para um objeto HugeInt
9  HugeInt::HugeInt( long value )
10 {
11     // inicializa array em zero
12     for ( int i = 0; i < digits; i++ )
13         integer[ i ] = 0;
14
15     // coloca dígitos do argumento no array
16     for ( int j = digits - 1; value != 0 && j >= 0; j-- )
17     {
18         integer[ j ] = value % 10;
19         value /= 10;
20     } // fim do for
21 } // fim do construtor default/conversão HugeInt
22
23 // construtor de conversão que converte uma string de caracteres
24 // representando um inteiro grande para um objeto HugeInt
25 HugeInt::HugeInt( const string &number )
26 {
27     // inicialize array em zero

```

Figura 19.24 ■ Definições de função-membro e função friend da classe HugeInt. (Parte 1 de 3.)

```

28     for ( int i = 0; i < digits; i++ )
29         integer[ i ] = 0;
30
31     // coloca dígitos do argumento no array
32     int length = number.size();
33
34     for ( int j = digits - length, k = 0; j < digits; j++, k++ )
35         if ( isdigit( number[ k ] ) ) // garante que o caractere é um dígito
36             integer[ j ] = number[ k ] - '0';
37 } // fim do construtor de conversão HugeInt
38
39 // operador de adição; HugeInt + HugeInt
40 HugeInt HugeInt::operator+( const HugeInt &op2 ) const
41 {
42     HugeInt temp; // resultado temporário
43     int carry = 0;
44
45     for ( int i = digits - 1; i >= 0; i-- )
46     {
47         temp.integer[ i ] = integer[ i ] + op2.integer[ i ] + carry;
48
49         // determina se transportará 1
50         if ( temp.integer[ i ] > 9 )
51         {
52             temp.integer[ i ] %= 10; // reduz para 0-9
53             carry = 1;
54         } // fim do if
55         else // sem carry
56             carry = 0;
57     } // fim do for
58
59     return temp; // retorna cópia do objeto temporário
60 } // fim da função operator+
61
62 // operador de adição; HugeInt + int
63 HugeInt HugeInt::operator+( int op2 ) const
64 {
65     // converte op2 para um HugeInt, depois chama
66     // operator+ para dois objetos HugeInt
67     return *this + HugeInt( op2 );
68 } // fim da função operator+
69
70 // operador de adição;
71 // HugeInt + string que representa grande valor inteiro
72 HugeInt HugeInt::operator+( const string &op2 ) const
73 {
74     // converte op2 para um HugeInt, depois chama
75     // operator+ para dois objetos HugeInt
76     return *this + HugeInt( op2 );
77 } // fim de operator+
78
79 // operador de saída sobrecarregado
80 ostream& operator<<( ostream &output, const HugeInt &num )
81 {
82     int i;
83
84     for ( i = 0; ( num.integer[ i ] == 0 ) && ( i <= HugeInt::digits ); i++ )
85         ; // pula zeros iniciais

```

Figura 19.24 ■ Definições de função-membro e função friend da classe HugeInt. (Parte 2 de 3.)

```

86
87     if ( i == HugeInt::digits )
88         output << 0;
89     else
90         for ( ; i < HugeInt::digits; i++ )
91             output << num.integer[ i ];
92
93     return output;
94 } // fim da função operator<<

```

Figura 19.24 ■ Definições de função-membro e função friend da classe HugeInt. (Parte 3 de 3.)

```

1 // Figura 19.25: fig19_25.cpp
2 // Programa de teste de HugeInt.
3 #include <iostream>
4 #include "Hugeint.h"
5 using namespace std;
6
7 int main()
8 {
9     HugeInt n1( 7654321 );
10    HugeInt n2( 7891234 );
11    HugeInt n3( "99999999999999999999999999999999" );
12    HugeInt n4( "1" );
13    HugeInt n5;
14
15    cout << "n1 é " << n1 << "\nn2 é " << n2
16         << "\nn3 é " << n3 << "\nn4 é " << n4
17         << "\nn5 é " << n5 << "\n\n";
18
19    n5 = n1 + n2;
20    cout << n1 << " + " << n2 << " = " << n5 << "\n\n";
21
22    cout << n3 << " + " << n4 << "\n= " << ( n3 + n4 ) << "\n\n";
23
24    n5 = n1 + 9;
25    cout << n1 << " + " << 9 << " = " << n5 << "\n\n";
26
27    n5 = n2 + "10000";
28    cout << n2 << " + " << "10000" << " = " << n5 << endl;
29 } // fim do main

```

[illegible]

Figura 19.25 ■ Inteiros imensos.

19.10 Classe *RationalNumber*. Crie uma classe `RationalNumber` (frações) com as seguintes capacidades:

- a) Crie um construtor que impeça um denominador 0 em uma fração, reduza ou simplifique frações que não estejam na forma reduzida e evite denominadores negativos.
- b) Sobrecarregue os operadores de adição, subtração, multiplicação e divisão para essa classe.
- c) Sobrecarregue os operadores relacionais e de igualdade para essa classe.

19.11 Classe *Polynomial*. Desenvolva a classe `Polynomial`. A representação interna de um `Polynomial` é um array de termos. Cada termo contém um coeficiente e um expoente, por exemplo, o termo

$$2x^4$$

tem o coeficiente 2 e o expoente 4. Desenvolva uma classe completa que contenha as funções construtor e destrutor apropriadas, além de funções *set* e *get*. A classe também deverá fornecer as seguintes capacidades de operador sobrecarregado:

- a) Sobrecarregue o operador de adição (+) para somar dois `Polynomials`.
- b) Sobrecarregue o operador de subtração (-) para subtrair dois `Polynomials`.
- c) Sobrecarregue o operador de atribuição para atribuir um `Polynomial` a outro.
- d) Sobrecarregue o operador de multiplicação (*) para multiplicar dois `Polynomials`.
- e) Sobrecarregue o operador de atribuição com adição (+=), o operador de atribuição com subtração (-=) e o operador de atribuição com multiplicação (*=).